

Received 4 February 2025, accepted 19 March 2025, date of publication 31 March 2025, date of current version 11 April 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3556313

RESEARCH ARTICLE

Rethinking Technological Investment and Cost-Benefit: A Software Requirements Dependency Extraction Case Study

GOURI GINDE¹, (Member, IEEE), GUENTHER RUHE^{1,2}, (Senior Member, IEEE), AND CHAD SAUNDERS³

¹Department of Electrical and Software Engineering, University of Calgary, Calgary, AB T2N 1N4, Canada

²Department of Computer Science, University of Calgary, Calgary, AB T2N 1N4, Canada

³Haskayne School of Business, University of Calgary, Calgary, AB T2N 1N4, Canada

Corresponding author: Gouri Ginde (gouri.ginde@ucalgary.ca)

ABSTRACT Machine Learning (ML) is widely used for different purposes within Software Engineering. It can substantially improve the efficiency and effectiveness of organizations. While various methods and techniques exist, all of them have strengths and weaknesses under varying scenarios and contexts. Thus far, the selection and implementation of ML techniques rely almost exclusively on accuracy criteria. This narrow perspective ignores crucial considerations of anticipated costs of the ML activities versus the projected benefits gained from applying the results. Thus, in this study we introduce a return-on-investment (ROI) perspective to evaluate ML techniques in Software Engineering, offering a novel lens to assess their true value beyond traditional benchmarks. We present findings for an approach that addresses this gap by enhancing the accuracy criterion with return on investment (ROI) considerations. Specifically, we extract dependencies from textual descriptions of software requirements and analyze the performance of two state-of-the-art ML techniques: Random Forest and Bidirectional Encoder Representations from Transformers (BERT), a encoder only Large Language Model. Drawing upon two publicly available data sets, we compare decision-making based on 1) exclusively on accuracy and 2) on ROI analysis to provide decision support for the selection and usage of ML classification methods. As such, our results showed that, 1) chasing model accuracy improvisation through increased annotated data does not generate expected returns in traditional ML methods. 2) For complex ML algorithms, the need for larger annotated dataset investment cost is justified by the higher returns, however, the trade-offs between accuracy and ROI become evident.

INDEX TERMS Machine learning, return on investment, requirements dependency extraction, management decision-making, BERT, random forest.

I. INTRODUCTION

The rapid rise in digitization, access to big data and growing technology and data analytics have revolutionized data-driven decision-making. Many firms look at managing their business more effectively using their data assets. Companies target a return of \$3.50 for every dollar spent on big data and analytics projects. However, many projects only yield returns of \$0.50, which leaves room for improvement and the possibility that organizations will push back against

new analytic technologies [51]. Thus, return on investment (ROI) is crucial for decision-making in both engineering and business, which is valid for Software Engineering (SE). Mainly because when choosing an ML method (among conventional and newer resource and data-intensive LLMs), beyond accuracy performance measure, it is crucial to determine the cost of ML choice and benefit (past break-even point) of technological investment before further exploration.

Software Engineering (SE) is characterized by various uncertainties. Being highly human-centric, SE involves processes that are not strictly repeatable, and the goals and constraints of software development are constantly

The associate editor coordinating the review of this manuscript and approving it for publication was Yin Zhang¹.

changing. Under such circumstances, experimentation and Data Analytics (DA) become inherently challenging. Figure 1 illustrates a typical ROI (cost-benefit) curve for technology usage, showing that after a certain point of growth, the curve reaches saturation, indicating that further investment does not yield additional benefits. We hypothesize that a similar pattern applies to DA [15].

There exists an extensive variety of ML algorithms and this pool is growing steadily. A recent study [48] listed Decision Trees, Naive Bayes, and Random Forrest as the techniques most frequently applied in Software Engineering. However, it is important to determine which algorithm works well for a given problem and which are less effective. The performance of any ML technique is generally measured in terms of accuracy (such as F1-score, F2) and precision and recall. However, the success of ML does not only depend on the algorithms used because ML is a process with various interdependent steps and the investments made in this process need to be related to the return gained from its results.

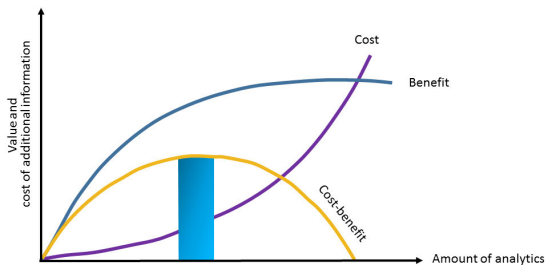


FIGURE 1. Break-even point from cost-benefit analysis of technology investment [15] shows that after a certain point of growth, the curve reaches saturation, indicating that further investment does not yield additional benefits.

Our research hypothesis is that ROI-driven DA helps identify the break-even point of investment, thereby optimizing the resources allocated to this process. ML includes methods, tools, and techniques for inferring models from data and has provided successful applications of classification and prediction algorithms. In the area of software development and evolution, a recent study [48] revealed that there is a spectrum of applications of ML across the software development life-cycle, with most of the applications belonging to the category of *Quality Assurance and Analytics*.

For example, for Software Requirements Dependency Extraction (RDE) using ML for automation poses several challenges such as [26]: 1) Due to the nature of the problem, it is cognitively difficult to identify all the dependencies among requirements; hence generating or procuring high-quality annotations for automation through ML is an arduous task. 2) In the real-world, unlabelled data is abundant and supervised ML techniques need a training dataset. 3) Textual requirements lack structure due to natural language, and feature extraction (transformation of the raw text into suitable internal numerical representations i.e. feature vector) techniques of NLP lead to ML techniques' success. However, feature extraction method identification and application are

cost and effort-intensive. 4) While there is a broad spectrum of ML techniques to choose from for automation, not all techniques are economically viable in all the scenarios considering data size and effort investment. Hence, there is a need to evaluate the ML techniques beyond just performance measures for effective decision making.

In this research, we present multiple empirical studies of a problem from requirements engineering. While it serves as one sample topic for a broader problem, Requirements Dependency Classification (RDC) has been a topic of interest on its own for both researchers and practitioners. Deshpande and Ruhe [17] report the results of a recent survey for requirements dependency classification and maintenance, with 76% of responses (53 out of 70) from practitioners. More than 80% of the participants agreed or strongly agreed that dependency extraction is cognitively difficult. Dependency information has implications for maintenance, and ignoring dependencies has a significant impact on project success [17].

For requirements dependency extraction, we study a fine-tuned *BERT* (Bidirectional Encoder Representations from Transformers) [19] method, a widely used encoder only Large Language model (LLM) [1], [30], [33]. BERT is a technique published by researchers from Google. We compare BERT with Random Forest (RF), which was selected from a traditional comparison between three widely used machine learners being Naive Bayes, Support Vector Machine (SVM), and RF.

The objective of this study is to support decision-making for the selection and usage of ML classification methods. Most studies compare machine learners by the performance in terms of accuracy. We expand this perspective by also looking at the ROI of method usage. The two perspectives result in different recommendations on which method is preferable in what situation.

This work extends our former conference publication [16], and a workshop paper [18], thesis [13] with (i) additional research questions, (ii) comprehensive empirical results, (iii) a detailed modeling of the ROI based on a workflow model of ML classification, and (iv) interpretation of results towards decision support. and ROI tool paper [61] We make following contributions in this research work:

- **Introduction of a enhanced ROI framework:** We propose a return-on-investment (ROI) framework for evaluating the cost-benefit balance of ML (ML) technologies in Software Engineering, shifting the focus from purely technical metrics to economic value.
- **Application to dependency extraction:** The framework is applied to a case study on software requirements dependency extraction, showcasing its utility in assessing both the effectiveness and economic value (cost-benefit) of ML models.
- **Empirical evaluation of ML techniques:** Through a systematic evaluation, we analyze the performance of multiple ML techniques in extracting dependencies from requirements and quantify their associated costs.

- Comparison with baseline approaches: We benchmark ML-based approaches against traditional methods, providing insights into the trade-offs between accuracy, scalability, and ROI.
- Practical implications for stakeholders: The study highlights actionable recommendations for practitioners and decision-makers, illustrating how ROI considerations can guide investment decisions in SE technologies.
- Replication and reproducibility: To encourage further research, we provide a comprehensive replication package, including datasets, code, and evaluation metrics, ensuring transparency and enabling other researchers to extend our work.

The remainder of the paper is structured as follows: Section II presents our research questions in the context of requirements dependency classification and ROI in software engineering management. Section III elaborates on our ROI modeling of the ML process, the data and research context employed, and our specific approach to addressing the research questions followed by our findings in section IV. The discussion in section V details the implications and limitations of this study before summarizing our conclusions in section V-D.

II. BACKGROUND

In this section we explore various preliminaries and background concepts of our study.

A. REQUIREMENTS DEPENDENCY CLASSIFICATION

Requirements for a system describe functionality of a service or what the system should do and the constraints on its operation [11]. Dependencies among requirements are concerned with the relationships between requirements and their impact on various activities in the software development process [11]. There are a variety of dependency types such as *Requires*, *And*, *Or* and *Precedence*. The term “dependency” can have a different connotation in implementation space, feature efforts space, feature value space and feature usage space [4], [45]. Requirements dependency extraction is the process of identifying the relationship and its type between a pair of requirements. Similar to requirements elicitation [37], extracting requirements dependencies is a cognitively difficult problem. These dependencies not only influence software development but also impact how requirements operate [24].

Figure 2 illustrates the practical relevance of considering requirements dependencies for incremental and iterative software development. We consider three consecutive releases called R_{i-1} , R_i , R_{i+1} . The sequence of releases defines the order of implementing and testing new or updated features, i.e., features of R_{i-1} are handled before features of R_i . However, suppose a requirement x is implemented in a release R_i but semantically requires another requirement y which actually is supposed to be implemented in a later release. In that case, requirement x will not be usable. Similar arguments hold for two requirements that are related to

each other but are implemented in different releases. Thus, identifying the dependencies helps to reduce the likelihood of a working solution that does not require costly rework.

B. RETURN ON INVESTMENT IN SOFTWARE ENGINEERING

ROI, also referred to as cost-benefit analysis has been explored intensively in decision making. Farbey and Finkelstein [22] explained that as a product moves through its life cycle, various evaluation methods such as ROI, multi-objective multi-criteria and value analysis play an important role in decision-making and recommended ROI as a strategy to decrease uncertainty in the business area and to improve the knowledge of how technology operates.

In a case study of a large telecommunication software system, Khoshgoftaar et al. [32] demonstrated a methodology for cost-benefit analysis of a software quality classification model. The cost and benefit computations were based on the type-I (FP) and type-II (FN) values of classification models. However, the authors did not consider the time and effort investment needed for cost computation. In another study on calculating ROI for software product lines [6], cost and benefit estimates based on organization-level criteria, such as cost to the organization and cost of reuse were considered. However, this analysis did not involve the costs and return of data analytics of any form.

In analyzing the ROI of investing in improving software quality, scholars crystallized the issue by noting that fixing existing software takes valuable time and resources away from developing new software [21]. But how much investment in software quality is desirable? When should we invest, and where? While these issues are difficult to quantify, they are crucial for the success of software-based products.

Begel and Zimmermann [3] gathered data from a survey of 200 Microsoft developers and testers, using 145 questions deemed relevant for Data Analytics (DA) at Microsoft which were posed to each participant and asked, “How important is it to have a software DA team answer this question?” with the expected answer on a five-point scale (*Essential to I don’t understand*). Although this analysis provides a sneak peek at Microsoft’s designing, development, planning, building, testing, and deploying very large software products and testing environments, it did not emphasize any form of ROI. Most of the survey questions included investigated the quality of many different proxy metrics for measuring the productivity of software developers and understanding the relationship between individual productivity and team or product success. Essentially, we speculate that the ROI aspect was softened into asking for the perceived subjective importance through this question.

Boehm et al. [7], [8] presented quantitative results on the ROI of Systems Engineering based on the analysis of the 161 software projects in the COCOMO II database and provided quantitative evidence that doing a minimal job of software systems engineering significantly reduces

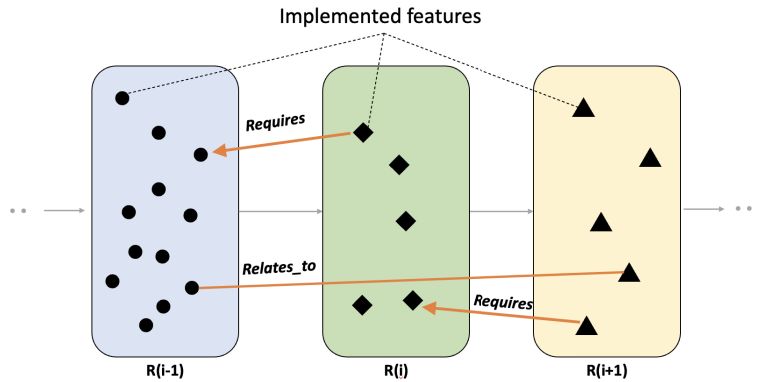


FIGURE 2. Requirements dependencies across various releases of project: Every release will have requirements which might have dependency of the type *Requires* or *Relates-to* with other requirements that are planned in different releases.

project productivity. Thus, ROI computation which include business or mission cost, schedule, and performance that could surface as a result of inadequate systems architecting and risk resolution provide more clarity for decision making. Ruhe and Nayebe [43] proposed the *Analytics Design Sheet* as a means to design the main goal and components of the DA process. The sheet is qualitative, while ROI analysis goes further and adds a quantitative perspective for outlining DA.

Ling et al. [38] proposed a system to predict the escalation risk of current defect reports for maximum ROI based on mining historic defect report data from an online repository. ROI was computed by estimating the cost of not correcting an escalated defect (false negative) to be seven times the cost of correcting a non-escalated defect (false positive).

Ferrari et al. [25] studied the ROI for text mining and showed that it has a tangible impact in not only terms of ROI but also intangible benefits, which arise from the investment in the knowledge management solution. This solution translates the returns directly that must be considered while integrating the financial perspective of analysis with the non-financial ones.

Weiss and Tian [59] emphasized how the quality of external data influences the results, and quantified the effort of gathering and using such data when it is available at a premium, into cost in terms of CPU time, even though the treatment of the subject is limited to a static setting. Similarly, Nagrecha and Chawla [41] proposed a Net Present Value model to determine the cost and impact of analytics programs for an organization. However, this approach does not consider data pre-processing cost as an additional cost aspect and transforms ML metrics into dollar amounts to arrive at cost and benefits.

Several empirical studies have explored diverse computational methods that used natural language processing (NLP) [10], [47], semi-supervised technique [14], hybrid techniques [15] and deep learning [28] in this context. Precision, Recall, F1-score (i.e., measure of model's accuracy, harmonic mean of precision and recall), and AUC were by far the most

common performance measures used by researchers in these papers. However, none of the approaches considered ROI to decide among techniques and the depth and breadth of their execution level.

Recognizing the contributions and limitations of prior work, we developed our research questions to consider both the data pre-processing costs as an additional cost aspect and transformed ML metrics into dollar amounts, using derived costs and benefits validated by industry experts.

III. RESEARCH DESIGN AND METHOD

Conventional ML methods such as Random forest, Naive Bayes and state-of-the-art encoder only LLMs such as BERT have been explored extensively in SE domain [48]. In this study we utilize these widely used methods to evaluate our research idea. Thus, we formulated the research question as follows.

A. RESEARCH QUESTIONS (RQs)

We formulate four RQs in this study. The first two RQs explore comparison based on the performance for the two chosen methods RF and RDC-BERT exclusively. RQ3 and RQ4 compare and evaluate the two methods for the two datasets (Firefox and Typo3) using performance measures F1 and ROI. **Comparison based on different criteria:**

RQ1: How does RF's performance compare in terms of F1 and ROI for the two datasets, Firefox and Typo3?

Rationale: We studied two open-source software (OSS) datasets. Firefox, a software application from Mozilla family [39] and Typo3 from [56], a content management software. Our goal is to evaluate two extraction techniques to demonstrate the impact of the consideration of ROI and accuracy considerations.

RQ2: How does RDC-BERT's (fine-tuned BERT for RDC) performance compare in terms of F1 and ROI for the datasets Firefox and Typo3?

Rationale: For two OSS datasets, we analyze RDC-BERT and compare F1 and ROI at different training set sizes.

Comparison between methods:

RQ3: How do RDC-BERT and RF compare in terms of F1 for the two datasets

Rationale: We evaluate two open-source software (OSS) datasets for RDC-BERT and RF to demonstrate the impact of the consideration of the F1-score.

RQ4: How do RDC-BERT and RF compare in terms of ROI for the two datasets?

Rationale: We evaluate two OSS datasets for RDC-BERT and RF to demonstrate the impact of the consideration of ROI in addition to accuracy considerations.

B. ML PROCESS MODELING

ML classification is an iterative process comprising a series of steps, and various ML workflows have been defined in the literature [2], [23]. A typical approach includes four Phases: Planning, Data Preparation, Execution, and Validation. Depending on the context, the effort allocated for these steps may vary, however, our approach parallels the process steps, and guidelines for pragmatic optimization in software engineering by [44]. Figure 3 illustrates an overview of the steps. We did not show all the possible arrows to indicate that loops can and do occur between any two steps in the process.

Phase A: Planning - Scoping and problem formulation (Step 1) define the problem context and its boundaries. The problem formulation eventually needs to be adjusted, as asking the right question constitutes the greatest effort for any application effort. Evaluation of candidate machine learners (Step 2) helps to evaluate various ML algorithms such as linear regression, logistic regression and K-means. We used term frequency inverse document frequency (TF-IDF) to generate word vectors before training. TF-IDF is a feature extraction technique that is widely used weighting metrics for measuring the relationship of words to documents. For RF, we used TF-IDF to generate word vectors before training. Also, hyper-parameter tuning was performed, and the results for 10-fold cross-validation were computed, followed by testing.

Phase B: Data Preparation - Data collection (Step 3) considers what is potentially relevant and checks the type and availability of the data. Data pre-processing (Step 4) transforms raw (unstructured) data into useful data for error-free results. Performing such pre-processing operations, for example, data cleaning, normalization, and feature extraction are essential for the success of ML classification, but these steps consume a considerable amount of human resources and processing time. The outcome of data pre-processing is the train set which could be processed through ML algorithm further [35]. Labeling (Step 5) assigns labels to ground truth data [2]. Labeling is generally performed by domain experts who identify a set of samples to train the ML models.

Phase C: Execution - The key idea of ML is to learn from existing data (Step 6) and then apply the resulting model to new data. The quality and quantity of the train set are often

as important as the actual ML algorithm. Hyper-parameter tuning (Step 7) means achieving settings that enable good algorithmic performance [36]. After training, the model is applied to the selected test set(s) (a small part of labeled data that is held out and excluded from the training process) (Step 8).

Phase D: Validation - Success from Step 8 does not automatically imply the success of the results in the context of the application. The external validity (Step 9) of the problem formulation and the data might prevent the applicability of the results (i.e., not actionable within the organization resulting in significant wasted effort). Internal validity measures how well a study is conducted (its structure) and is also the extent to which a research study establishes a trustworthy cause-and-effect relationship. This type of validity depends largely on the study's procedures and how rigorously it is performed and how accurately its results reflect the studied group. Internal validation approaches such as cross-validation cannot guarantee the quality of an ML model due to potentially biased training data. External validation is critical for evaluating the generalizability of the ML model, where independently derived datasets (external) are leveraged as validation datasets. While such independent validation is also sometimes used to refer to a validation study by researchers other than the researchers who developed the model [29].

Cost estimation is known to be inherently difficult in software engineering [49], [55], and this holds true for valid for benefit prediction. However, despite many factors influencing the costs and benefits of ML classifications, we provide a preliminary model to allow an estimate of the ROI. For cost estimation, we assume that the cost of performing ML classification with any given ML technique is the sum of the cost components of the four phases outlined above. To simplify the model, we focus on Phase B (Data Preparation) and Phase C (Execution) and exclude the other two phases for the purposes of analysis. Finally, we assume an 80:20 effort (and cost) ratio between Phase B and Phase C, emphasizing that most effort is spent on data preparation [63].

For modeling the benefit of the classification results, we look at classification errors, and their cost (penalty) created. A *confusion matrix* (CM) is a matrix that contains information relating actual with predicted classifications. For n classes, CM will be an $n \times n$ matrix associated with a classifier. Table 1 shows the principal entries of CM for binary classification. The F1 score measures the model's accuracy based on the training set and is defined as the harmonic mean of the model's precision and recall in equation (3) and F2 is a another score which has the effect of lowering the importance of precision and increase the importance of recall thus emphasizing impact of FN in score computation.

$$\text{Precision}(i) = \frac{TP(i)}{TP(i) + FP(i)} \quad (1)$$

$$\text{Recall}(i) = \frac{TP(i)}{TP(i) + FN(i)} \quad (2)$$

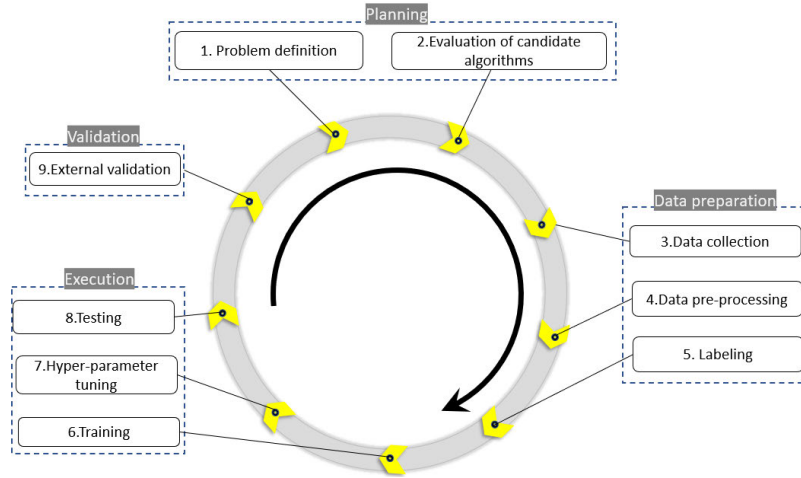


FIGURE 3. Overview of the nine steps constituting the ML process.

$$F1(i) = \frac{2 \times \text{Precision}(i)\text{Recall}(i)}{\text{Precision}(i) + \text{Recall}(i)} \quad (3)$$

$$F2(i) = \frac{5 \times \text{Precision}(i)\text{Recall}(i)}{4 \times \text{Precision}(i) + \text{Recall}(i)} \quad (4)$$

$TP(i)$: # of requirement pairs truly classified as category i

$FP(i)$: # of requirement pairs falsely classified as category i

$FN(i)$: # of requirement pairs falsely classified as not being category i

In the context of dependency classification, the benefit could be modeled as the ability of the ML model to produce the least amount of overhead by 1) incorrectly classifying independence as a dependency (FP) and 2) incorrectly classifying dependency as independent (FN). So, using $Cost_{FP}$ and $Cost_{FN}$ as estimated re-work costs due to classification overhead, $Sum(Cost_{FN}, Cost_{FP})$ would be the cumulative expense that a company incurs. In a release cycle, if *estimated value* that a product could generate is: $Value_{prod}$ then the *Benefit* would be the difference of the *estimated value* and the *classification overhead*.

During every classification, *Cost* and *Benefit* were computed using various *Cost factors* such as data processing costs (Phase B and Phase C) for all the train (N_{train}) and test (N_{test}) samples (n) in every iteration. This is further translated into dollar-cost by multiplying by hourly charges (C_{HR}) of N_{HR} human resources.

$$Cost = n \times \sum_{\text{all applicable}} Cost\ factors \times N_{HR} \times C_{HR} \quad (5)$$

Return computations for RDC assume penalty ($Cost_{FP}$) for misidentifying the independent requirements (FP) and penalizing ($Cost_{FN}$) instances that were falsely identified as independent (FN).

$$TotalPenalty = FP \times Cost_{FP} + FN \times Cost_{FN} \quad (6)$$

$$Benefit = Value_{prod} - TotalPenalty \quad (7)$$

Return and investment are context-specific terms, and studying the ROI of ML classification needs tailoring to the

context of the study. To determine the ROI, we follow the simplest form of its calculation relating to the difference between *Benefit* and *Cost* to the amount of *Cost* as shown in (8). Both *Benefit* and *Cost* are measured as human effort in person-hours.

$$ROI = (Benefit - Cost) / Cost \quad (8)$$

While more exotic formulations of ROI were considered, given the dual objectives of providing a common language that both technical and business staff could easily understand and for ease of operationalization in practice, a simple formulation was preferred. The core investigative focus of our study is to evaluate various conditions under which RDC-BERT (fine-tuned BERT using data specific to requirement dependency extraction) is preferable to Random Forrest (RF).

In this empirical analysis, beginning with a small training set, classifiers were created, and then the training set size was incremented slowly by a fixed factor to generate new classifiers in every iteration until all the data available for training was exhausted. In every iteration, the classifiers were tested for a small fixed data set to capture the results.

C. PROBLEM FORMULATION

While there are different types of dependencies between requirements [9], [62], we provide the definitions just for the ones used in our empirical study. As the data comes from different real-world projects, we followed the definitions used in the practical context.

For a set of requirements R and a pair of requirements $(r, s) \in R \times R$

- 1) Two requirements r, s are called **INDEPENDENT** if handling one of them has no logical or practical implication for handling the other one. Otherwise, they are called **DEPENDENT**.
- 2) **REQUIRES** is a form of **DEPENDENT** relationship. If requirement r requires the requirement s to be implemented, then r and s are in a **REQUIRES** relationship. **REQUIRES** is an asymmetric relationship.

TABLE 1. A confusion matrix of binary (two) class classification problem.

	Predicted Negative	Predicted Positive
Actual Negative	True Negative (TN)	False Positive (FP)
Actual Positive	False Negative (FN)	True Positive (TP)

- 3) *RELATES_TO* is another specific form of *DEPENDENT* relationship. Requirement *r* relates_to requirement *s* if changing one impacts the other. *RELATES_TO* is a symmetric relationship.¹

Problem definition: Binary Requirements Dependency Classification (RDC) - For a given set *R* of requirements and their textual description, we consider two binary RDC problems. They are called RDC1 and RDC2, respectively.

RDC1 is to decide for a given pair $(r,s) \in R \times R$ if (r,s) is in a *REQUIRES* relationship.

RDC2 is to decide for a given pair $(r,s) \in R \times R$ if (r,s) is in a *RELATES_TO* relationship.

D. RESEARCH CONTEXT AND DATA

Online bug tracking systems such as Bugzilla [40], and Redmine [42] are widely used in open-source software development. Feature requests, tasks, bugs, epics, stories, features, enhancements, and new requirements are logged into these systems in the form of issue reports [5], [50], which help software developers to track these items for effective implementation [52], testing and release planning [46]. We mined data from Bugzilla and Redmine repositories using their respective REST APIs. Firefox is a Mozilla web browser application, and Typo3 is a content management system.

Firefox: In Bugzilla, feature requests are specific issues typically tagged as “enhancement” [39]. We retrieved these feature requests for the Firefox project using the search engine in the Bugzilla issue tracking system. We exported all the related fields such as Title, Type, Priority, Product, Depends_on, and Blocks. Each issue report contains dependency relationships with other issue reports as references metadata [34]. Using this information, 3,773 depends_on (also interpreted as *REQUIRES* dependency type) requirements pairs were retrieved. To generate negative samples, requirements that had no relationship were paired, and 21,358 samples were generated.

Typo3: Typo3 [56] is a free and open-source web-based management and issue-tracking tool website. It allows users to manage multiple independent and associated projects. Numerous issues across a range of projects are updated each day which helps software developers to track these items for effective implementation. In Redmine, features are a specific type of issue that is extracted in this paper for further data analysis. Typo3: Content Management System (CMS) is an Open Source Enterprise Content Management System [56] with a large global community of approximately 900 members of the TYPO3 Association. We collected

information such as issue_links, description, the version found, the version released, issue_id etc., for 5,017 features using Redmine’s REST API through a Python script.

All feature descriptions with fewer than three words were filtered out, resulting in 1,324 feature pairs with dependency type *RELATES_TO*. Using the rest of the features that were not dependent on others, 9,270 pairs were generated as a negative sample set. Table 2 map sample pairs of requirements dependencies. For example, to be able to associate the address with a payment card *REQUIRES* ability to use a new billing address when adding a new payment card. For both data sets, both positive and negative samples are needed for training to perform binary classification. Since we only had dependent (positive) samples in the data, we generated negative samples by pairing the unrelated requirements in the given snapshot of the dataset.

Typo3, currently at released version 11, is a complex content management system developed as a hybrid OSS software product. It has a core team of 12 members with varying skills and expertise. They have a major release cycle of 18 months, and they plan two or more releases ahead of time. Developers are encouraged to track the dependencies in Jira; however, a few team members utilize post-its to work and track them. Typo3 does not explicitly consider Requirements Engineering as a development phase, but they term the efforts towards identifying features and extracting dependencies as conceptual work or scoping. Over 15% of the release, the cycle is identified as scoping effort, and about 25% of scoping in a release cycle is identified as dependency extraction and identification. Nine team members and the CTO are involved, mostly in identifying the dependencies.

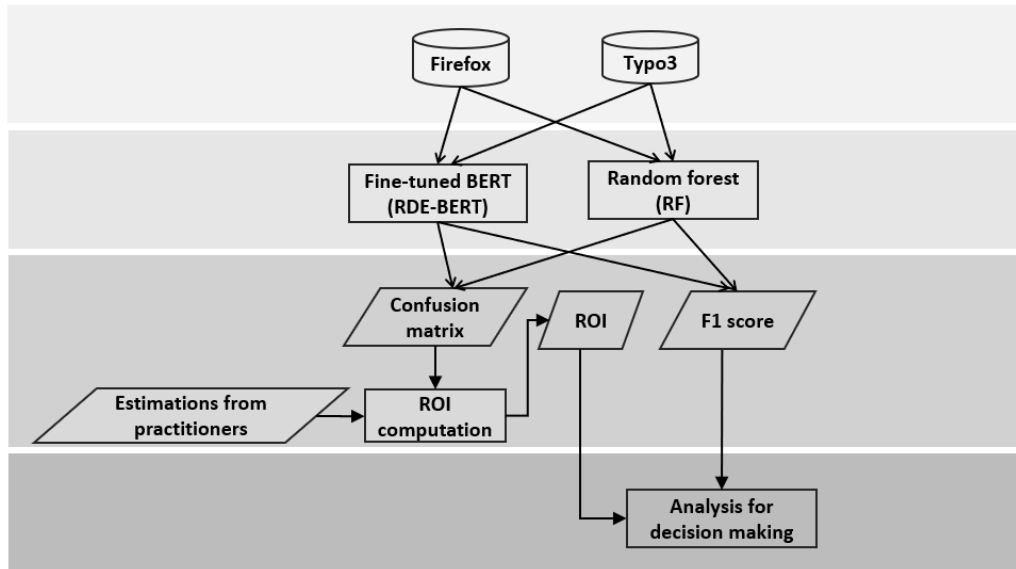
The CEO confirmed that about 80% of the features are in some form dependent on each other, and missing the dependencies is more problematic than misidentifying them. As the CEO stated, “...if you miss dependencies, it starts to ramp up quickly, which is when things go wrong and break deadlines. We wanted to release in April (4 weeks ago) now the deadline is mid-October”.

Typo3 identifies and manages seven dependencies such as precedes, blocks and clones and caused_by. Most of the dependency issues are identified through testing, and the estimated re-work is about 12%. Minimal manual testing is needed as test suits of over 75,000 test cases are used. The CEO estimated that the overwork caused by missing dependencies is about 10% of the efforts. The average salary of the nine people involved in re-work is \$70 (CAD). Based on the effort, resource hourly charges and timelines information gathered from the key informant practitioner, we computed and derived the estimates for FN (cost of missing dependency) and FP (cost of misidentifying

¹There are other types of dependencies such as *DUPLICATES*, *BLOCKS* etc. that also occur in these datasets; however, we have considered the ones that occur most frequently.

TABLE 2. Samples of dependency pairs from the two datasets. *REQUIRES* dependency pairs are from Firefox and *RELATES_TO* from Typo3 dataset.

Dependency type	Requirement 1	Requirement 2
<i>REQUIRES</i>	add the ability to associate saved billing address with payment card in add/edit card form	option to use new billing address when adding new payment card
	update illustration for error connection failure	ux error connection failure copy design and illustration update
<i>RELATES_TO</i>	ignore button for link targets	make it possible to mark specific links to not get checked by linkvalidator
	page tree filter: make it possible to explicitly filter by uid	advanced filtering for the page-tree

**FIGURE 4.** Overview of the experiment setup that utilizes two datasets, two different methodologies and various evaluation metrics for comparison.**TABLE 3.** Parameter settings for the two empirical analysis scenarios.

Parameters	Values
Phase B	1.5 min/sample
Phase C	0.30 min/sample
C_{HR}	\$70/hr
N_{HR}	10
N	Firefox: 7,546
	Typo3: 2,648
$Cost_{FN}$	\$25,000
$Cost_{FP}$	\$10,000
$Value_{prod}$	\$4,000,000

*We consider ratio of Phase B & Phase C as 80:20 [64]

independency). Also $Value_{prod}$ was computed using these cost estimates for a period of one release cycle (= 18 months). These estimates are summarized in Table 3.

E. EXPERIMENTAL DESIGN

Figure 4 provides an overview of our experimental setup. The complete approach is multi-layered, as denoted by the shading. Each of these layers could be further expanded to include additional elements for solution space evaluation.

In this study, to generate the results, RF, Naive Bayes and SVM ML algorithms were compared against RDC-BERT for the two datasets: Firefox and Typo3. Overall eight experiments were conducted. Since RF performed better among all the conventional ML algorithms [16], we report the results of RF and RDC-BERT (i.e., a total four experiments). For each experiment, we computed the ROI using the False Negative and False Positive values from the confusion matrix. We summarize the analyses conducted by RQ and the output generated in Table 4.

Requirements pairs were pre-processed to eliminate noise, such as spatial characters and numbers. The generated output is fed into RDC-BERT and RF for training. Care was taken to process the same data snapshot through RF and RDC-BERT models. Further, the fine-tuned BERT model (RDC-BERT) was then used for classification. The data was split (80:20) into train and test sets and balanced between classes.

We conducted classification by utilizing a fraction of the whole dataset for training and testing for a small fixed data set. This was repeated by slowly increasing the training set, and capturing the results.

RDC-BERT: For fine-tuning BERT, a pre-trained BERT model is used in combination with our RDC specific dataset.

TABLE 4. Overview of the analyses conducted by RQ.

		Description	
Fig 5	F1_ROI_RF_Firefox	Firefox: F1 vs ROI of RF	RQ 1
Fig 6	F1_ROI_RF_Typo3	Typo3: F1 vs ROI of RF	RQ 1
Fig 7	F1_ROI_RDC-BERT_Firefox	Firefox: F1 vs ROI of RDC-BERT	RQ 2
Fig 8	F1_ROI_RDC-BERT_Typo3	Typo3: F1 vs ROI of RDC-BERT	RQ 2
Fig 9	F1_Firefox	Firefox: Compare F1 of RF and RDC-BERT	RQ 3
Fig 10	F1_Typo3	Typo3: Compare F1 of RF and RDC-BERT	RQ 3
Fig 11	ROI_Firefox	Firefox: Compare ROI of RF and RDC-BERT	RQ 4
Fig 12	ROI_Typo3	Typo3: Compare ROI of RF and RDC-BERT	RQ 4

The result is a fine-tuning BERT model called RDC-BERT. To fine-tune the BERT model, we used *NextSentencePrediction*,² a sentence pair classification pre-trained BERT model, and further fine-tuned it for the RDC specific dataset on Tesla K80 GPU on Google Colab.³

In every instance, for a given training set size, RDC-BERT was trained through three epochs with a batch size of 32 and a learning rate of 2e-5. In each epoch, the training set was divided into 90% for training and 10% for validation. Finally, RDC-BERT was used to classify the test set, and the resulting F1-score and confusion matrix was captured. BERT eliminates the need for feature extraction since it is a language model based on deep learning. BERT, pre-trained on a large text corpus, can be fine-tuned on specific tasks by providing only a small amount of domain-specific data.

IV. FINDINGS

In this section, we report the findings to address our research questions based upon an analysis for the two data sets introduced above and the two techniques under investigation using estimates from Table 3. In this analysis, we considered one technique at a time from the perspective of both the F1-score and ROI, in order to support decision-making regarding the question of when increasing accuracy no longer pays off.

A. COMPARISON OF F1-SCORES AND ROI FOR RF (RQ1)

For Firefox, for RF, in Figure 5: F1_ROI_RF_Firefox shows that F1 and ROI for the training set size lower than 40% is better than that of RDC-BERT. Chasing a higher F1 score does not pay off, and a closer look at the benefits versus investment for additional training data was needed. Interestingly, at 40% train set, F1-score=0.7 for both RDC_BERT and RF.

For Typo3, Figure 6 shows that for up to 15% of the train set, ROI remains negative and picks up only beyond 20% of the train set. Max ROI=48 is reached with 70% train set and F1=0.8. However, ROI begins to drop beyond this, although F1-score continues to improve. This implies that chasing the F1 score beyond a certain point might not generate expected returns (i.e., diminishing returns).

²https://huggingface.co/transformers/model_doc/bert.html#bertfornextsentenceprediction

³<https://colab.research.google.com/>

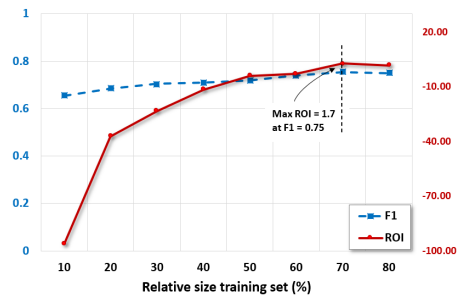


FIGURE 5. F1 vs ROI of RF for Firefox dataset, utilizing values from Table 3.

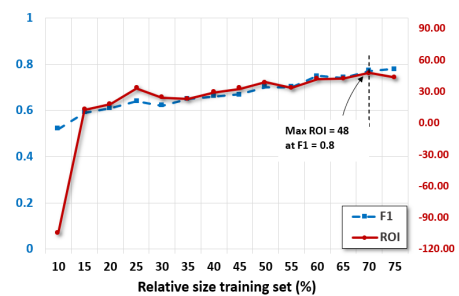


FIGURE 6. F1 vs ROI of RF for Typo3 dataset, utilizing values from Table 3.

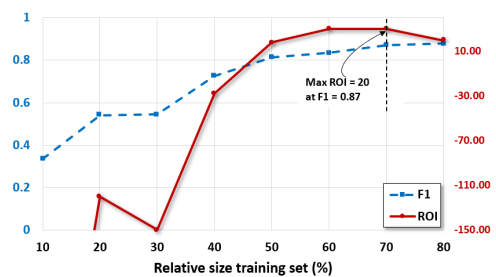


FIGURE 7. F1 vs ROI of RDC-BERT for Firefox dataset, utilizing values from Table 3.

B. COMPARISON OF F1-SCORES AND ROI FOR RDC-BERT (RQ2)

For Firefox, Figure 7: F1_ROI_RDC-BERT_Firefox shows that for RDC-BERT, F1-score increases linearly; however, max ROI is achieved when the train set is 70% of the dataset.

For Typo3, Figure 8: F1_ROI_RDC-BERT_Typo3 shows that F1-score and ROI grow steeply for RDC-BERT with the increasing training set size. However, similar to Firefox, ROI and F1 of RF is stable and better than RDC-BERT

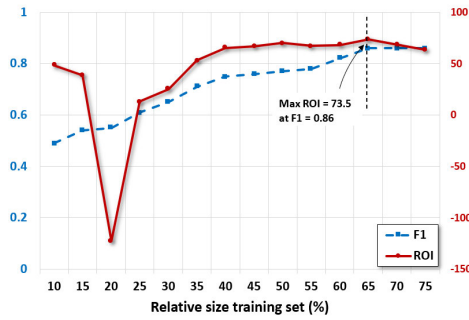


FIGURE 8. F1 vs ROI of RDC-BERT for Typo3 dataset, utilizing values from Table 3.

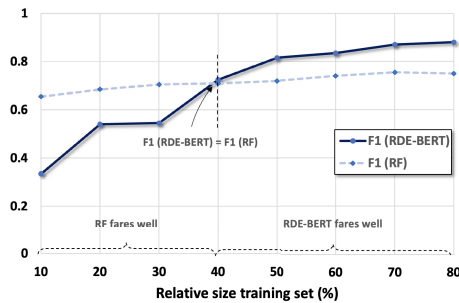


FIGURE 9. F1 of RDC-BERT vs RF for the Firefox dataset.

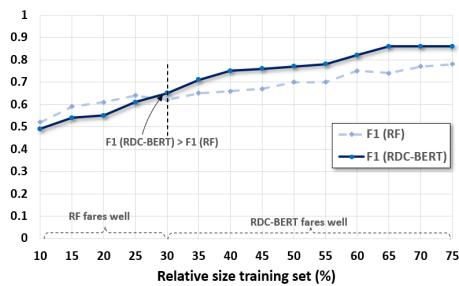


FIGURE 10. F1 of RDC-BERT vs RF for the Typo3 dataset.

for the training set smaller than 30%. These findings again emphasize the need to reconsider how F1 and ROI together could aid in decision-making when selecting an ML approach. When comparing RDC-BERT with RF using ROI criteria (Fig 8 and 9) across the two data sets, RF outperforms RDC-BERT for the lower training set size (incurring lower negative returns). However, positive ROIs are observed only in the more extensive training set at which RDC-BERT is consistently better than RF. These findings clearly illustrate the trade-offs between the F1 and ROI for both datasets.

C. COMPARISON OF F1-SCORES FOR RF AND RDC-BERT (RQ3)

The traditional approach for comparing techniques is to look at just accuracy for some fixed training set. Figures 9 (F1_Firefox) and 10 (F1_Typo3) show the comparison of the F1-scores for varying training set sizes for the two datasets. Results show that RF achieves a higher accuracy more quickly for small training sets. However, with a training set greater than 40% of the dataset for Firefox and

30% for Typo3, RDC-BERT achieves better results overall. As illustrated in figures 9 and 10, an increased training set does not yield a better F1-score beyond 65%. The F1-score hits a plateau and even starts to degrade for both of the methods and datasets.

In both datasets studied, it was evident that RDC-BERT models require large amounts of data (at least 30% or more) to stabilize and show value (steady positive ROI). Based upon the Firefox findings (Fig 10 and 11), RDC-BERT approaches the 80% benchmark accuracy with approximately 50% of the training data. At the same time, RF requires 70% training data to attain the same level of accuracy. However, both techniques can achieve positive ROI with as little as 50% training data. However, RDC-BERT achieves maximum ROI (30) with an accuracy of 0.87 with approximately 70% training data, while RF achieves maximum ROI (= 2.2) with an accuracy of 0.75 with approximately 70% training data.

D. COMPARISON OF ROI OF RF AND RDC-BERT (RQ4)

Comparison of ROI for the two datasets and two methods (RDC-BERT and RF) is shown in Figures 11 (ROI_Firefox) and 12 (ROI_Typo3) respectively.

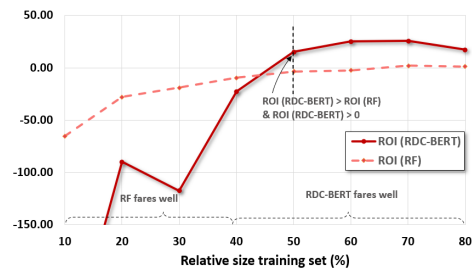


FIGURE 11. ROI of RDC-BERT vs RF for Firefox dataset.

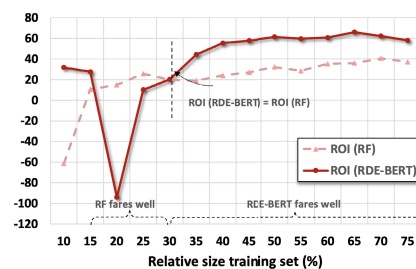


FIGURE 12. ROI of RDC-BERT vs RF for the Typo3 dataset.

With a smaller training set for Firefox, RF again performs better comparatively, even though the ROI is negative. Similar results are evident for Typo3. RF performs marginally better ROI-wise for the smaller training set. ROI of RDC-BERT picks up pace only beyond 40% and 30% training set size for Firefox and Typo3, respectively.

As illustrated in figures 9 and 10, an increased training set does not yield a better F1-score beyond 65%. The F1-score hits a plateau and even starts to degrade for both of the methods and datasets. Based upon the Typo3 findings (Fig 10 and 12), RDC-BERT approaches the 80% benchmark

TABLE 5. Various metrics for the relative size set for RF and RDC-BERT methods for Firefox dataset.

Relative size	Random Forest					RDC-BERT				
	ROI	F1	F2	Prec	Rcl	ROI	F1	F2	Prec	Rcl
10	-118.84	0.655	0.66	0.64	0.67	-506.85	0.335	0.24	1.00	0.20
20	-47.25	0.685	0.70	0.65	0.72	-147.98	0.54	0.56	0.50	0.59
30	-29.93	0.705	0.72	0.68	0.73	-179.18	0.545	0.46	0.77	0.42
40	-15.47	0.71	0.73	0.67	0.76	-33.88	0.725	0.70	0.71	0.61
50	-6.08	0.72	0.75	0.66	0.79	21.20	0.815	0.83	0.79	0.84
60	-4.37	0.74	0.76	0.71	0.78	34.98	0.835	0.88	0.76	0.92
70	2.48	0.755	0.78	0.71	0.80	35.84	0.87	0.90	0.83	0.92
80	1.27	0.75	0.78	0.70	0.81	23.45	0.88	0.89	0.86	0.90

accuracy with approximately 55% of the training data, while RF requires 70% training data to reach the same level of accuracy. However, RBC-BERT and RF can achieve positive ROI with as little as 15% training data. However, RBC-BERT achieves maximum ROI (73.5) with an accuracy of 0.86 with approximately 65% training data, while RF achieves maximum ROI (48) with an accuracy of 0.80 with approximately 70% training data. Thus, RBC-BERT can deliver higher ROI and similar levels of accuracy than RF for the same amount of training data.

Table 5 provides all the different metrics captured in these experiments with incremental datasets for Firefox project. Similar to F1 other measures (F2, Precision and Recall) show relative behaviour barring Precision of RDC-BERT which shows slight increase in the precision beyond 80% of the train set. Since all the results have been accumulated after five runs, we speculate continuing the tests for few more times might be beneficial for concrete conclusion on more finer metrics such as Precision and Recall.

V. DISCUSSION

In this section we discuss various implication of our study, and identify the limitations and measures taken to address them.

A. IMPLICATIONS

ML is not simply a cost of doing business, rather it is a foundational activity that can provide value for the money invested. Our proposed approach aligns this notion with the strategic direction of the organization. While return on investment (ROI) is a common approach used for business planning and decision making, it is not applied as widely within software engineering or specifically within applied ML.

In our study, we demonstrate how to instantiate ROI in the context of RDC. Our approach provides a pragmatic link between the business and technical aspects of the organization by providing a common language that incorporates both the technical aspects inherent in the evaluation of accuracy, with the business considerations of costs and benefits. We argue that this is an extremely powerful approach that provides evidence that is compelling and consistent for both technical and business decision-making.

In addition, we think that the ROI approach could sensitize the ML team to the entire process of ML classification and

how that process fits into organizational processes. The ROI approach is essential for evaluating the possible tradeoffs between accuracy and the benefits. Mainly because without consideration of the key dependencies within the process, benefits in one part of the process (e.g., improved accuracy) can easily be undermined by excessive costs in another part of the process that would not typically be considered if focused exclusively on accuracy. Alternatively, lower levels of accuracy in the ML process might be acceptable if other benefits are accruing at reasonable costs. Thus, valuable ML investments are potentially being avoided based upon not meeting accuracy expectations, when those ML solutions could be sufficient to realize high payoffs for the organization.

Our approach increases the transparency of the decision-making process by adding diversity to the evaluation criteria that foreground the various trade-offs being made. The development of AI tools that businesses and consumers can trust is essential for their continued adoption, especially as there is increasing regulatory scrutiny of the biases that arise in the ML algorithms or inherent in the data used for training.

While ML algorithms are generally trusted for relatively mechanical well-defined problems, this trust plummets when the decisions are subjective, and likely to vary by contextual variables that are not well understood. This in turn increases the pressure to adjust ML algorithms for variations in specific markets further driving up development costs. Such pressure directs the focus on customizing products and services based upon ML algorithms for specific markets while increasing costs further and undermines the benefits for certain markets or customers [27]. The proposed approach considers technical and business aspects simultaneously and provides a more traceable set of interconnected processes. This approach includes business and technical considerations to enable management to evaluate the risks of some undesirable decisions and the trade-offs needed to realize the likely benefits.

B. LIMITATIONS AND FUTURE WORK

We have explored RF and RDC-BERT in the context of the RDC domain, the applicability to other domains and ML algorithms remains to be explored. However, we anticipate that our general approach is applicable to other domains the rely upon data that require significant processing before

TABLE 6. Summary of the findings by product, relative train set size, and performance measures for RF and RDC_BERT.

Product	Train set size	Performance measure	RF	RDC BERT
Typo3	15%, 20%, 25%	F1	🏆	
		ROI	🏆	
	30%	F1		🏆
		ROI	Draw	Draw
	35% and above	F1		🏆
		ROI		🏆
Firefox	10%, 20%, 30%	F1	🏆	
		ROI	🏆	
	40%	F1	Draw	Draw
		ROI	🏆	
	50% and above	F1		🏆
		ROI		🏆

ML algorithms can be applied. Similarly, there is nothing particularly unique about RF or BERT that could not be applied to other established ML techniques or those developed in-house, although caution is warranted. Hardware and computational costs are not included to simplify the model and in recognition that these costs decline significantly when amortized at scale, but this assumption would need to be validated within each domain. Although we have taken care to randomize the data by shuffling and using stratified split to take care of balanced data in both training and validation, multiple runs with varying first iteration data samples are needed to be more confident in the conclusions made. However, we argue that the key observations made are valid and robust based upon the empirical validation and robustness checks performed. Finally, while our experimental design provided the opportunity to control for key factors influencing the processes under investigation and thus increase internal validity, this is done in part by trading external validity. However, by seeding our model with estimates and key assumptions validated by senior executives, we hope to increase the applicability to industry, although additional work is needed.

We use RDC as a sandbox to build proof of the concept based on the two ML techniques used to solve RDC. Evaluating other techniques such as GBM/XGBoost and state-of-the-art LLM techniques such as GPT models [58], [60] [31] remains as part of the future work. In the future, we will also extend the results along various dimensions. For example, contacting practitioners who can evaluate our prototype and help us gather the cost factors of true positive (i.e., effort and cost saving due to automation using ML) would help us enhance our ROI model further. With a broader dataset and knowledge base, the findings from this work could lead to a customized recommendation system to support practitioners' decision-making regarding the tradeoffs between technical and business criteria in evaluating how much data analytics is enough. In light of the ongoing development in this direction, we have developed a proof of concept tool⁴ that can process various textual data

sets to provide such trade-off analysis, and work is in progress to extend it to additional modalities such as image data sets.

C. BEYOND ROI

Securing the data needed for training an ML algorithm often needs human effort for retrieval, pre-processing, and annotation. The benefits gained from the application of ML can be dramatically offset due to data collection and data pre-processing activities, which incur substantial costs and require extensive effort [22]. However, there are various other cost factors such as computation (GPU/CPU usage), due to ever evolving state-of-the-art Large Language Models such as Llama and ChatGPT, [41], monitoring and sustainability costs (carbon footprints) [20], classifier confidence [54], and prediction robustness which need to be considered further while computing ROI, which is out side of scope for this work.

While ROI can provide value projection of ML-based solution approach, other considerations such as responsible AI (bias, explainability, reliability, security and transparency) are important when dealing with data with human aspects, in the realm of Requirements Engineering and Software Engineering in general. This is also because AI/ML-based software solutions is incremental and iterative which calls for continuous development. Thus, there is never a final version of a ML solution to a given problem. Rather it needs to be updated and improved continuously over a period either by feeding new data into the system or by retraining new versions of the system [57]. This trait of ML-solutions implies that the ethical issues posed by the implementations need continuous monitoring and measurement.

Utilizing ML-based approach to solve SE problems that involve human aspects is not a black-and-white task; instead, it is a spectrum of these biases that might accidentally be built into the solution which need to be tackled in every iteration through additional data and (or) ML algorithm selection. However, gathering more data on the potential biases is more costly and an investment. Pragmatically, there must be a limit to how far such investments can be made to achieve a certain degree of responsible AI. Thus, there is a trade-off between responsible AI aspects and the ROI of chosen AI-

⁴<https://roi.shaktiilab.org/>

based solution approach. We envision to explore this aspect in our future work.

D. CONCLUSION

In this study, we demonstrate that in the context of AI initiatives based upon ML, just looking at performance measures such as accuracy could lead to short-term decisions that fail to recognize the long-term payoffs. Conversely, ignoring the cost and benefit of such a classification potentially risks an emphasis on improving accuracy that might not generate any value for the additional effort. We provide a high-level ML process for classification (i.e., supervised ML), however, with minimal changes, this process can easily be adapted to unsupervised or semi-supervised ML methods. Going forward, the selection or development of ML algorithms will need to receive the same care and attention that current decisions over recruiting a new manager from outside the firm or developing a manager within the firm does today.

With growing interest in integrating AI/ML in almost all domains, the focus must shift towards responsible and trustworthy AI/ML-based solutions [12], [53]. However, responsible AI software solutions are not a black-and-white task; instead, it is a spectrum of concerns such as fairness, reliability, safety, security, and explainability that might accidentally be built into the software, which need to be tackled in every iteration through additional data and (or) ML algorithm selection. Thus, gathering more data on the potential biases is more costly and an investment. Pragmatically, there must be a limit to how far such investments can be made to achieve a certain degree of responsible AI. Hence, there is a trade-off between responsible AI aspects and the return on investment of the chosen AI-based solution approach. In future work, we will develop methods to evaluate trade-offs between responsible AI-related aspects and return on investment, which will help determine the extent to which AI/ML adds value while abiding by the standards of responsible AI.

REFERENCES

- [1] M. Ajagbe and L. Zhao, "Retraining a BERT model for transfer learning in requirements engineering: A preliminary study," in *Proc. IEEE 30th Int. Requirement Eng. Conf. (RE)*, Aug. 2022, pp. 309–315.
- [2] S. Amershi, A. Begel, C. Bird, R. DeLine, H. Gall, E. Kamar, N. Nagappan, B. Nushi, and T. Zimmermann, "Software engineering for machine learning: A case study," in *Proc. IEEE/ACM 41st Int. Conf. Softw. Eng., Softw. Eng. Pract. (ICSE-SEIP)*, May 2019, pp. 291–300.
- [3] A. Begel and T. Zimmermann, "Analyze this! 145 questions for data scientists in software engineering," in *Proc. 36th Int. Conf. Softw. Eng.*, May 2014, pp. 12–23.
- [4] O. Ben-Assuli, T. Heart, R. Klempfner, and R. Padman, "Human-machine collaboration for feature selection and integration to improve congestive heart failure risk prediction," *Decis. Support Syst.*, vol. 172, Sep. 2023, Art. no. 113982.
- [5] T. Bhowmik and S. Reddivari, "Resolution trend of just-in-time requirements in open source software development," in *Proc. IEEE Workshop Just-Time Requirement Eng. (JITRE)*, Aug. 2015, pp. 17–20.
- [6] G. Bockle, P. Clements, J. D. McGregor, D. Muthig, and K. Schmid, "Calculating ROI for software product lines," *IEEE Softw.*, vol. 21, no. 3, pp. 23–31, May 2004.
- [7] B. Boehm, L. Huang, A. Jain, and R. Madachy, "The ROI of software dependability: The idave model," *IEEE Softw.*, vol. 21, no. 3, pp. 54–61, May 2004.
- [8] B. Boehm, R. Valerdi, and E. Honour, "The ROI of systems engineering: Some quantitative results for software-intensive systems," *Syst. Eng.*, vol. 11, no. 3, pp. 221–234, Sep. 2008.
- [9] P. Carlshamre, K. Sandahl, M. Lindvall, B. Regnell, and J. N. O. Dag, "An industrial survey of requirements interdependencies in software product release planning," in *Proc. 5th IEEE Int. Symp. Requirements Eng.*, Aug. 2001, pp. 84–91.
- [10] J. Natt och Dag, B. Regnell, P. Carlshamre, M. Andersson, and J. Karlsson, "A feasibility study of automated natural language requirements analysis in market-driven development," *Requirements Eng.*, vol. 7, no. 1, pp. 20–33, Apr. 2002.
- [11] Å. G. Dahlstedt and A. Persson, "Requirements interdependencies: State of the art and future challenges," in *Engineering and Managing Software Requirements*. Cham, Switzerland: Springer, 2005, pp. 95–116.
- [12] F. Dehghani et al., "Trustworthy and responsible AI for human-centric autonomous decision-making systems," 2024, *arXiv:2408.15550*.
- [13] G. Deshpande, "Requirements dependency extraction: Advanced machine learning approaches and their ROI analysis," Ph.D. dissertation, Dept. Comput. Sci., Univ. Calgary, Calgary, AB, Canada, 2022.
- [14] G. Deshpande, C. Arora, and G. Ruhe, "Data-driven elicitation and optimization of dependencies between requirements," in *Proc. IEEE 27th Int. Requirements Eng. Conf. (RE)*, Sep. 2019, pp. 416–421.
- [15] G. Deshpande, Q. Motger, C. Palomares, I. Kamra, K. Biesialska, X. Franch, G. Ruhe, and J. Ho, "Requirements dependency extraction by integrating active learning with ontology-based retrieval," in *Proc. IEEE 28th Int. Requirements Eng. Conf. (RE)*, Aug. 2020, pp. 78–89.
- [16] G. Deshpande and G. Ruhe, "Beyond accuracy: ROI-driven data analytics of empirical data," in *Proc. 14th ACM / IEEE Int. Symp. Empirical Softw. Eng. Meas. (ESEM)*, Oct. 2020, pp. 1–6.
- [17] G. Deshpande and G. Ruhe. (Dec. 2020). *Survey: Elicitation and Maintenance of Requirements Dependencies*. [Online]. Available: <https://ispma.org/elicitation-and-maintenance-of-requirements-dependencies-a-state-of-the-practice-survey/>
- [18] G. Deshpande, B. Sheikhi, S. Chakka, D. L. Zotegouon, M. N. Masahati, and G. Ruhe, "Is BERT the new silver bullet?—An empirical investigation of requirements dependency classification," in *Proc. IEEE 29th Int. Requirements Eng. Conf. Workshops (REW)*, Sep. 2021, pp. 136–145.
- [19] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," 2018, *arXiv:1810.04805*.
- [20] L. Dixon, J. Li, J. Sorensen, N. Thain, and L. Vasserman, "Measuring and mitigating unintended bias in text classification," in *Proc. AAAI/ACM Conf. AI, Ethics, Soc.*, Dec. 2018, pp. 67–73.
- [21] H. Erdogmus, J. Favaro, and W. Strigel, "Return on investment," *IEEE Softw.*, vol. 21, no. 3, pp. 18–22, Mar. 2004.
- [22] B. Farbey and A. Finkelstein, "Evaluation in software engineering: ROI, but more than ROI," *Tech. Rep.*, 2001.
- [23] U. Fayyad, G. Piatetsky-Shapiro, and P. Smyth, "The KDD process for extracting useful knowledge from volumes of data," *Commun. ACM*, vol. 39, no. 11, pp. 27–34, Nov. 1996.
- [24] A. Ferrari and G. Ginde, "Handbook on natural language processing for requirements engineering: Overview," in *Handbook on Natural Language Processing for Requirements Engineering*. Springer, 2025, pp. 1–15.
- [25] M. Ferrari, "ROI in text mining projects," *WIT Trans. State-Art Sci. Eng.*, vol. 17, pp. 155–183, May 2005.
- [26] I. Fagalist, C. Elsner, J. Bosch, and H. H. Olsson, "An end-to-end framework for productive use of machine learning in software analytics and business intelligence solutions," in *Proc. Intr. Conf. Product-Focused Softw. Process Improvement*. Cham, Switzerland: Springer, Jan. 2020, pp. 217–233.
- [27] R. C. di Carlo François Candelon, "Ai regulation is coming: How to prepare for the inevitable," *Harvard Bus. Rev.*, Sep. 2021.
- [28] J. Guo, J. Cheng, and J. Cleland-Huang, "Semantically enhanced software traceability using deep learning techniques," in *Proc. IEEE/ACM 39th Int. Conf. Softw. Eng. (ICSE)*, May 2017, pp. 3–14.
- [29] S. Y. Ho, K. Phua, L. Wong, and W. W. B. Goh, "Extensions of the external validation for checking learned model interpretability and generalizability," *Patterns*, vol. 1, no. 8, Nov. 2020, Art. no. 100129.
- [30] K. Kaur and P. Kaur, "Improving BERT model for requirements classification by bidirectional LSTM-CNN deep model," *Comput. Electr. Eng.*, vol. 108, May 2023, Art. no. 108699.
- [31] R. Khojah, M. Mohamad, P. Leitner, and F. Gomes de Oliveira Neto, "Beyond code generation: An observational study of ChatGPT usage in software engineering practice," 2024, *arXiv:2404.14901*.

- [32] T. M. Khoshgoftaar, E. B. Allen, W. D. Jones, and J. P. Hudepohl, "Cost-benefit analysis of software quality models," *Softw. Quality J.*, vol. 9, no. 1, pp. 9–30, Jan. 2001.
- [33] D. Kıcı, G. Malik, M. Çevik, D. Parikh, and A. Bener, "A BERT-based transfer learning approach to text classification on software requirements specifications," in *Proc. Can. Conf. AI*, Jun. 2021, p. 4207.
- [34] B. Kim, S. Kang, and S. Lee, "A weighted PageRank-based bug report summarization method using bug report relationships," *Appl. Sci.*, vol. 9, no. 24, p. 5427, Dec. 2019.
- [35] S. Ktsiantis, D. Kanellopoulos, and P. Pintelas, "Data preprocessing for supervised learning," *Int. J. Comput. Sci.*, vol. 1, no. 12, pp. 4104–4109, Dec. 2007.
- [36] M. Kuhn and K. Johnson, *Applied Predictive Modeling*, vol. 26. Cham, Switzerland: Springer, 2013.
- [37] S. Lim, A. Henriksson, and J. Zdravkovic, "Data-driven requirements elicitation: A systematic literature review," *Social Netw. Comput. Sci.*, vol. 2, no. 1, pp. 1–35, Feb. 2021.
- [38] C. X. Ling, S. Sheng, T. Bruckhaus, and N. H. Madhavji, "Predicting software escalations with maximum ROI," in *Proc. 5th IEEE Int. Conf. Data Mining (ICDM)*, Sep. 2005, pp. 717–720.
- [39] *Bugfields*. Accessed: Apr. 5, 2020. [Online]. Available: <https://bugs.documentfoundation.org/page.cgi?id=fields.html>
- [40] *Bugzilla*. Accessed: Apr. 5, 2020. [Online]. Available: <https://www.bugzilla.org/>
- [41] S. Nagrecha and N. V. Chawla, "Quantifying decision making for data science: From data acquisition to modeling," *EPJ Data Sci.*, vol. 5, no. 1, p. 27, Dec. 2016.
- [42] *RedmineIssueTrackingSetup—Redmine*. Accessed: Apr. 5, 2025. [Online]. Available: <https://www.redmine.org/projects/redmine/wiki/RedmineIssueTrackingSetup>
- [43] G. Ruhe and M. Nayeibi, "What counts is decisions, not numbers—Toward an analytics design sheet," in *Perspectives on Data Science for Software Engineering*. Amsterdam, The Netherlands: Elsevier, 2016, pp. 111–114.
- [44] G. Ruhe, "Optimization in software engineering: A pragmatic approach," in *Contemporary Empirical Methods in Software Engineering*. Cham, Switzerland: Springer, 2020, pp. 235–261.
- [45] G. Ruhe, *Product Release Planning: Methods, Tools and Applications*. Boca Raton, FL, USA: Auerbach Publications, 2010.
- [46] G. Ruhe and M. O. Saliu, "The art and science of software release planning," *IEEE Softw.*, vol. 22, no. 6, pp. 47–53, Nov. 2005.
- [47] R. Samer, M. Stettinger, M. Atas, A. Felfernig, G. Ruhe, and G. Deshpande, "New approaches to the identification of dependencies between requirements," in *Proc. IEEE 31st Int. Conf. Tools Artif. Intell. (ICTAI)*, Nov. 2019, pp. 1265–1270.
- [48] S. Shafiq, A. Mashkoo, C. Mayr-Dorn, and A. Egyed, "Machine learning for software engineering: A systematic mapping," 2020, *arXiv:2005.13299*.
- [49] M. Shepperd, "Cost prediction and software project management," in *Software Project Management in a Changing World*. Springer, 2014, pp. 51–71.
- [50] L. Shi, C. Chen, Q. Wang, S. Li, and B. Boehm, "Understanding feature requests by leveraging fuzzy method and linguistic analysis," in *Proc. 32nd IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, Oct. 2017, pp. 440–450.
- [51] J. P. Shim, A. M. French, C. Guo, and J. Jablonski, "Big data and analytics: Issues, solutions, and ROI," *Commun. Assoc. Inf. Syst.*, vol. 37, no. 1, p. 39, 2015.
- [52] Y. Shin, J. H. Hayes, and J. Cleland-Huang, "Guidelines for benchmarking automated software traceability techniques," in *Proc. IEEE/ACM 8th Int. Symp. Softw. Syst. Traceability*, May 2015, pp. 61–67.
- [53] A. Sorathiya and G. Ginde, "Ethical software requirements from user reviews: A systematic literature review," 2024, *arXiv:2410.01833*.
- [54] A. Tambwekar, A. Maiya, S. Dhavala, and S. Saha, "Estimation and applications of quantiles in deep binary classification," *IEEE Trans. Artif. Intell.*, vol. 3, no. 2, pp. 275–286, Apr. 2022.
- [55] Y. Timmerman, R. Nasfi, G. De Tré, F. Pattyn, and A. Bronselaer, "Cost-based analysis of the impact of data completeness and representational consistency," *Decis. Support Syst.*, vol. 175, Dec. 2023, Art. no. 114044.
- [56] *The TYPO3 Project and Community—Open Source CMS*. Accessed: Jun. 6, 2021. [Online]. Available: <https://typo3.org/>
- [57] Z. Wan, X. Xia, D. Lo, and G. C. Murphy, "How does machine learning change software development practices?" *IEEE Trans. Softw. Eng.*, vol. 47, no. 9, pp. 1857–1871, Sep. 2021.
- [58] J. Wei, A.-L. Courbis, T. Lambolais, B. Xu, P. L. Bernard, and G. Dray, "Zero-shot bilingual app reviews mining with large language models," in *Proc. IEEE 35th Int. Conf. Tools Artif. Intell. (ICTAI)*, Nov. 2023, pp. 898–904.
- [59] G. M. Weiss and Y. Tian, "Maximizing classifier utility when there are data acquisition and modeling costs," *Data Mining Knowl. Discovery*, vol. 17, no. 2, pp. 253–282, Oct. 2008.
- [60] J. S. Yeow, M. E. Rana, and N. A. Abdul Majid, "An automated model of software requirement engineering using GPT-3.5," in *Proc. ASU Int. Conf. Emerg. Technol. Sustainability Intell. Syst. (ICETIS)*, Jan. 2024, pp. 1746–1755.
- [61] N. Zambare, J. Idoko, J. Acharya, and G. Ginde, "AROH: An interactive toolkit for estimating ROI of data analytics," in *Proc. IEEE 15th Annu. Commun. Workshop Conf. (CCWC)*, Jan. 2025, pp. 01162–01169.
- [62] H. Zhang, J. Li, L. Zhu, R. Jeffery, Y. Liu, Q. Wang, and M. Li, "Investigating dependencies in software requirements for change propagation analysis," *Inf. Softw. Technol.*, vol. 56, no. 1, pp. 40–53, Jan. 2014.
- [63] S. Zhang, C. Zhang, and Q. Yang, "Data preparation for data mining," *Appl. Artif. Intell.*, vol. 17, nos. 5–6, pp. 375–381, 2003.

GOURI GINDE (DESHPANDE) (Member, IEEE) is currently an Assistant Professor with the Department of Electrical and Software Engineering, University of Calgary. Her research focus is data analytics, applied data science in software engineering, and healthcare. Before transitioning to academia, she spent a decade in the information technology (IT) industry working with Hewlett-Packard (HP) Enterprise and HP Laboratories, Bengaluru, India, where she held various roles, including a Software Engineer and a Senior Software Engineer. She is also the Scientific Director of the Software Hub for AnalytiKS, Technology and Innovation Hub (SHAKTI) Laboratory, University of Calgary. The vision of her research laboratory is to solve software-centric problems that are usable in the real world. She actively seeks collaborations with the industry to bridge gaps between industry and academia and foster innovation.

GUENTHER RUHE (Senior Member, IEEE) is currently the Industrial Research Chair of software engineering of the University of Calgary, this is a joint position between the Department of Computer Science and the Department of Electrical and Software Engineering, University of Calgary. Before coming to Calgary, he was the Deputy Director of the Fraunhofer Institute for Experimental Software Engineering Fh IESE, Germany. He is the Founder and the CEO of Expert Decisions Inc., which is the University of Calgary spin-off company. His main research interests include product release planning, software project management, decision support, open innovation, data analytics, empirical software engineering, and search-based software engineering. He is a member of ACM. He won various awards, such as Alberta iCORE (Informatics Center of Research Excellence) Award, the IBM Research Award, and the Microsoft Research Software Engineering Innovation Award. Since 2016, he has served as the Editor-in-Chief for *Information and Software Technology* (Elsevier).

CHAD SAUNDERS received the B.Sc. degree in applied mathematics and the M.B.A. degree from the Memorial University of Newfoundland and the Ph.D. degree in management information systems from the Haskayne School of Business. He is currently an Associate Professor of entrepreneurship and innovation with the Haskayne School of Business, University of Calgary. His work is published in leading journals, including *IEEE TRANSACTIONS ON SOFTWARE ENGINEERING*, *Journal of Business Venturing Insights*, *Entrepreneurship Theory and Practice*, *Journal of Business Venturing*, *International Journal of Entrepreneurial Behavior and Research*, and *Research Policy*. His research interests include the support that entrepreneurs draw upon in starting and scaling their ventures, the role of disruptive technology in enabling and constraining ventures and understanding the policy needed to build more inclusive support ecosystems for entrepreneurs.

• • •